

Introdução - C

Prof. Wesley Nunes Gonçalves¹
wnunesgoncalves@gmail.com

¹Universidade Federal de Mato Grosso do Sul

12 de agosto de 2024

1 Arrays de caracteres - strings

2 Funções

3 Escopo de Variáveis

4 Bibliotecas

Definição de uma string

- String: sequência de caracteres adjacentes na memória do computador.
- As strings têm no elemento seguinte a última letra, um caractere '\0'.

```
1 char str[6] = "ABC";  
2
```

Atribuição

- string é um array, e a linguagem C não suporta a atribuição de um array para outro.

```
1 #include <stdio.h>
2
3 int main(){
4     char str1[20] = "Hello World";
5     char str2[20];
6
7     str1 = str2; //ERRADO!
8 }
9
```

```
1 #include <stdio.h>
2
3 int main(){
4     int i;
5     char str1[20] = "Hello World";
6     char str2[20];
7
8     for (i = 0; str1[i]!='\0'; i++)
9         str2[i] = str1[i];
10    str2[i] = '\0';
11 }
12
```

Lendo uma string do teclado

- A função `scanf()` lê apenas strings digitadas sem espaços, ou seja, palavras.

```
1 char str[20];  
2 scanf("%s", str);
```

- `gets`: leitura considerando todos os caracteres digitados (incluindo os espaços) até encontrar uma tecla enter.

```
1 char str[20];  
2 gets(str);
```

Lendo uma string do teclado

- A função `fgets()` recebe três parâmetros de entrada:
 - `str`: a string a ser lida.
 - `tamanho`: o limite máximo de caracteres a serem lidos.
 - `fp`: a variável que está associada ao arquivo de onde a string será lida.

```
1 char *fgets (char *str, int tamanho, FILE *fp);
```

- E retorna:
 - `NULL` : no caso de erro ou fim do arquivo.
 - O ponteiro para o primeiro caractere da string recuperada em `str`.

Lendo uma string do teclado

- A função `fgets()` especifica o tamanho máximo da string de entrada.

```
1 #include <stdio.h>
2
3 int main(){
4     char nome[30];
5
6     printf(" Digite um nome: ");
7     fgets(nome, 30, stdin);
8
9     printf("O nome digitado foi: %s", nome);
10    return 0;
11 }
12
```

Limpendo o buffer do teclado

- Às vezes, podem ocorrer erros durante a leitura de caracteres ou strings do teclado.
- Para resolver esses erros, podemos limpar o buffer do teclado antes de realizar a leitura de caracteres ou strings.
- Basicamente, a função `setbuf(stdin, NULL)` preenche um buffer (primeiro parâmetro) com determinado valor (segundo parâmetro).

```
1 #include <stdio.h>
2
3 int main() {
4     char string[100];
5     float a;
6
7     scanf("%s", string); // Digitar uma string compostas, ex: Alg e prog
8     //setbuf(stdin, NULL);
9     scanf("%f", &a);
10
11     printf(" String: %s\n", string);
12     printf(" Numero: %.2f\n", a);
13     return 0;
14 }
15
```


Funções para Manipular Strings

- Calculando o tamanho

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main(){
5     char str[15] = "teste";
6     printf("%zu", strlen(str));
7
8     return 0;
9 }
10
```

- Comparando strings

```
1 #include <stdio.h>
2 #include <string.h>
3 int main(){
4     char str1[100], str2[100];
5     printf("Entre com uma string: ");
6     gets(str1);
7     printf("Entre com uma string: ");
8     gets(str2);
9     int r = strcmp(str1, str2);
10    if(r == 0) {
11        printf("Strings iguais");
12    } else {
13        printf("Strings diferentes");
14    }
15    return 0;
16 }
17
```

Funções para Manipular Strings

● Copiando strings

```
1 #include <stdio.h>
2 #include <string.h>
3 int main(){
4     char str1[100], str2[100];
5     printf("Entre com uma string: ");
6     gets(str1);
7     strcpy(str2, str1);
8     printf("String 1: %s\n", str1);
9     printf("String 2: %s\n", str2);
10
11     return 0;
12 }
13
14
```

● Concatenando strings

```
1 #include <stdio.h>
2 #include <string.h>
3 int main(){
4     char str1[15] = "Bom ";
5     char str2[15] = "dia!";
6     strcat(str1, str2);
7     printf("%s\n", str1);
8
9     return 0;
10 }
11
```

Funções

- Função: sequência de comandos que recebe um nome e pode ser chamada de qualquer parte do programa, quantas vezes forem necessárias, durante a sua execução.
- Duas razões principais para o uso de funções:
 - Estruturação dos programas.
 - Reutilização de código.

```
1 tipo_retornado nome_função (lista_de_parâmetros){  
2     sequência de declarações e comandos  
3 }  
4
```

Funções

- Exemplo:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int square(int a){
5     return (a*a);
6 }
7
8 int main(){
9     int n1,n2;
10    printf("Entre com um numero: ");
11    scanf("%d", &n1);
12    n2 = square(n1);
13    printf("O seu quadrado vale: %d\n", n2);
14    return 0;
15 }
16
```

Funções

● Usando protótipo

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 //protótipo da função
4
5 int square (int a);
6
7 int main(){
8     int n1, n2;
9     printf("Entre com um numero: ");
10    scanf("%d", &n1);
11    n2 = square(n1);
12    printf("O seu quadrado vale: %d\n", n2);
13
14    return 0;
15 }
16
17 int square (int a){
18     return (a*a);
19 }
20
```

Funções

● Leitura de um menu

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int menu(){
5     int i;
6     do {
7         printf("Escolha uma opção:\n");
8         printf("(1) Opcao 1\n");
9         printf("(2) Opcao 2\n");
10        printf("(3) Opcao 3\n");
11        scanf("%d", &i);
12    } while ((i < 1) || (i > 3));
13    return i;
14 }
15
16 int main(){
17     int op = menu();
18     printf("Vc escolheu a Opcao %d.\n", op);
19
20     return 0;
21 }
22
```

Escopo de Variáveis

- O escopo é o conjunto de regras que determinam o uso e a validade das variáveis ao longo do programa, podendo ser **global** ou **local**.

```
1 #include <stdio.h>
2
3 int x = 5; //variável global
4
5 void incr(){
6     x++; //acesso a variavel global
7 }
8
9 int main(){
10     printf("x = %d\n", x); //acesso a variavel global
11     incr();
12     printf("x = %d\n", x); //acesso a variável global
13     return 0;
14 }
15
```

Escopo de Variáveis

```
1 #include <stdio.h>
2
3 void func1(){
4     int x = 10; // variavel local
5     printf("%d\n", x);
6 }
7
8 void func2(){
9     int x = 20; // variavel local
10    printf("%d\n", x);
11 }
12
13 int main(){
14     int x;
15     scanf("%d", &x);
16     printf("%d\n", x);
17     if(x == 5){
18         int y = 1;
19         printf("%d\n", y);
20     }
21     func1();
22     func2();
23     printf("%d\n", x);
24 }
25
```


Usando Bibliotecas

- O comando **#include** é usado para declarar as bibliotecas.
- Duas possibilidades de uso:
 - 1 **#include <nome_da_biblioteca>**: o pré-processador irá procurar pela biblioteca nos caminhos de procura pré-especificados do compilador.

Exemplo: Uso de uma biblioteca própria do sistema: `stdio.h` e `stdlib.h`.

Usando Bibliotecas

- O comando **#include** é usado para declarar as bibliotecas.
- Duas possibilidades de uso:
 - ① **#include <nome_da_biblioteca>**
 - ② **#include "nome_da_biblioteca"**: o pré-processador irá procurar pela biblioteca no mesmo diretório onde se encontra o código-fonte.

É possível ainda informar o nome do arquivo com o caminho completo.

Usando Bibliotecas

- O comando **#include** é usado para declarar as bibliotecas.
- Duas possibilidades de uso:
 - 1 **#include <nome_da_biblioteca>**
 - 2 **#include "nome_da_biblioteca"**

```
1 #include <stdio.h>
2 #include "D:\Programas\soma.h"
3
```

Criando Bibliotecas

- Código-fonte: main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int soma(int a, int b){
5     return a + b;
6 }
7
8 int subtracao(int a, int b){
9     return a - b;
10 }
11
12 int produto(int a, int b){
13     return a * b;
14 }
15
16 int divisao(int a, int b){
17     return a / b;
18 }
19
```

```
20 int main(){
21     int x,y,z;
22     char ch;
23     printf("Digite uma operacao (+,-,*,/): ");
24     ch = getchar();
25     printf("Digite 2 numeros: ");
26     scanf("%d %d", &x, &y);
27     switch(ch){
28         case '+': z = soma(x,y); break;
29         case '-': z = subtracao(x,y); break;
30         case '*': z = produto(x,y); break;
31         case '/': z = divisao(x,y); break;
32         default: z = soma(x,y);
33     }
34     printf("Resultado = %d\n", z);
35     return 0;
36 }
37
```

Criando Bibliotecas

- Para transformar as funções em uma biblioteca, precisamos de dois arquivos:
 - 1 **Cabeçalho da biblioteca:** esse arquivo contém as definições das funções (apenas o seu protótipo), tipos e variáveis da biblioteca. Sua extensão é `.h`.
 - 2 **Código-fonte da biblioteca:** arquivo que contém a implementação das funções definidas no cabeçalho. Sua extensão é `.c`.

Criando Bibliotecas

- Vamos chamar nossa biblioteca de aritmetica:

aritmetica.h

```
1 int soma(int a, int b);  
2 int subtracao(int a, int b);  
3 int produto(int a, int b);  
4 int divisao(int a, int b);  
5
```

Criando Bibliotecas

- Vamos chamar nossa biblioteca de aritmetica:

aritmetica.c

```
1 #include "aritmetica.h"
2
3 int soma(int a, int b){
4     return a + b;
5 }
6 int subtracao(int a, int b){
7     return a - b;
8 }
9 int produto(int a, int b){
10    return a * b;
11 }
12 int divisao(int a, int b){
13    return a / b;
14 }
15
```

Criando Bibliotecas

- Vamos chamar nossa biblioteca de aritmetica:

aritmetica.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "aritmetica.h"
4
5 int main(){
6     int x,y,z;
7     char ch;
8     printf(" Digite uma operacao (+,-,*,/): ");
9     ch = getchar();
10    printf(" Digite 2 numeros: ");
11    scanf("%d %d",&x, &y);
12    switch(ch){
13        case '+': z = soma(x,y); break;
14        case '-': z = subtracao(x,y); break;
15        case '*': z = produto(x,y); break;
16        case '/': z = divisao(x,y); break;
17        default: z = soma(x,y);
18    }
19    printf(" Resultado = %d\n",z);
20    return 0;
21 }
22
23
```